



Assignment Title:

C Programming Basics
Lab Report and Knowledge Check

Assignment Type:

Coursework : Lab Report

Module Name:

OS Security & Networks

Module Code:

ST5003CMD

Bsc (Hons) Computer Science With AI, Softwarica College of IT and E-commerce

Intake: September

Assignment Due: 20-Sept-2026

Submitted By:

CU:16537084

College ID: 250055

Name: Salman Ansari

Submitted To:

Name: Rupak Rajbanshi

Table of Contents:

I. ENVIROMENT SETUP.....	3
1. Setting Up the C-Programming Enviroment:.....	3
II. C PROGRAM STRUCTURE AND EXAMPLES.....	4
1. Basic Hello World Program.....	4
III. GCC COMPILATION PROCESS.....	5
A. Installing GCC on Linux.....	5
B. Four Stages of Compilation.....	5
TABLE I.....	5
THE FOUR STAGES OF C COMPILATION.....	5
C. COMPILATION STAGE.....	6
Program: 1.....	6
1. Stage 1: Preprocessing:.....	6
2. Stage 2: Compilation to Assembly:.....	6
3. Stage 3: Assembly to Object File:.....	6
4) Stage 4: Linking:.....	7
2. Program with User Input:.....	7
D. COMPILATION STAGE.....	8
Program: 2.....	8
1. Stage 1: Preprocessing:.....	8
2. Stage 2: Compilation to Assembly:.....	8
3. Stage 3: Assembly to Object File:.....	8
4. Stage 4: Linking:.....	9
3. Program with User Input:.....	9
E. COMPILATION STAGE.....	10
Program: 3.....	10
1. Stage 1: Preprocessing:.....	10
2. Stage 2: Compilation to Assembly:.....	10
3. Stage 3: Assembly to Object File:.....	10
4. Stage 4: Linking:.....	11
IV. RETURN VALUES AND EXIT STATUS.....	12
A.	12
i) Understanding Return Value.....	12
B. i) Checking Exit Status.....	12
Figure 19: Output & Return Value.....	12
ii) Understanding Return Value.....	13
ii) Checking Exit Status.....	13
Figure 21: Output & Return Value.....	13

V. RETURN VALUES AND EXIT STATUS.....	14
#. Understanding Concepts:.....	14
A. Question: 1.....	14
B. Question: 2.....	14
C. Question: 3.....	14
D. Question: 4.....	15
E. Question: 5.....	15
F. Question: 6.....	16

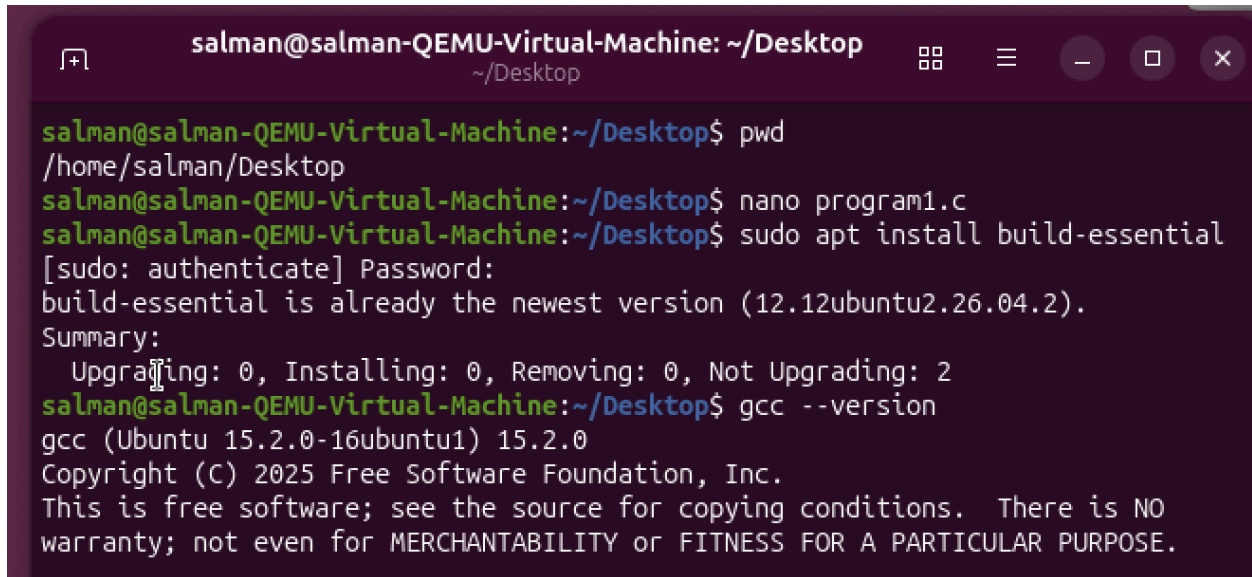
CONCLUSION:

Table of Figures:

Figure 1. Setting Enviroment.....	5
Figure 2. Basic Hello World Program.....	6
Figure 3. Install GCC.....	7
Figure 4. Preprocessing Command.....	8
Figure 5. Compilation Stage.....	8
Figure 6. Assembly.....	8
Figure 7. Linking Stage.....	9
Figure 8. User input.....	9
Figure 9: Preprocessing.....	10
Figure 10: Compilation.....	10
Figure 11: Assembly.....	10
Figure 12: Linking Command.....	11
Figure 13: Find multiple variable.....	11
Figure 14: Preprocessing.....	12
Figure 15: Compilation.....	12
Figure 16: Assembly.....	12
Figure 17: Linking and Run code.....	13
Figure 18: C-program.....	14
Figure 21: Output & Return Value.....	15
Figure 22: C-Program.....	17

I. ENVIROMENT SETUP

1. Setting Up the C-Programming Enviroment:

A terminal window titled 'salman@salman-QEMU-Virtual-Machine: ~/Desktop' with standard window controls. The terminal shows the following commands and output:

```
salman@salman-QEMU-Virtual-Machine:~/Desktop$ pwd
/home/salman/Desktop
salman@salman-QEMU-Virtual-Machine:~/Desktop$ nano program1.c
salman@salman-QEMU-Virtual-Machine:~/Desktop$ sudo apt install build-essential
[sudo: authenticate] Password:
build-essential is already the newest version (12.12ubuntu2.26.04.2).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 2
salman@salman-QEMU-Virtual-Machine:~/Desktop$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

Figure 1. Setting Enviroment

II. C PROGRAM STRUCTURE AND EXAMPLES

1. Basic Hello World Program

- The following demonstrates a basic C program structure with preprocessor directives, main function, and return statement:

Program 1

```
#include <stdio.h>

int main() {
    printf("Hello, World!\n")
    return 0;
}
```

Figure 2. Basic Hello World Program

III. GCC COMPILATION PROCESS

A. Installing GCC on Linux

```
salman@salman-QEMU-Virtual-Machine:~/Desktop$ pwd
/home/salman/Desktop
salman@salman-QEMU-Virtual-Machine:~/Desktop$ nano program1.c
salman@salman-QEMU-Virtual-Machine:~/Desktop$ sudo apt install build-essential
[sudo: authenticate] Password:
build-essential is already the newest version (12.12ubuntu2.26.04.2).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 2
salman@salman-QEMU-Virtual-Machine:~/Desktop$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

Figure 3. Install GCC

B. Four Stages of Compilation

TABLE I

THE FOUR STAGES OF C COMPILATION

Stage	Command	Output	Purpose
1. Preprocessing	gcc -E hello.c -o hello.i	.i file	Expand headers & macros
2. Compiling	gcc -S hello.i -o hello.s	.s file	Generate Assembly code
3. Assembling	gcc -c hello.s -o hello.o	.o file	Create object file
4. Linking	gcc hello.o -o hello	executable	Final executable

C. COMPILATION STAGE

Program: 1

1. Stage 1: Preprocessing:

```
salman@salman-QEMU-Virtual-Machine:~/Desktop$ gcc -E program1.c -o program1.i
salman@salman-QEMU-Virtual-Machine:~/Desktop$ cat program1.i
# 0 "program1.c"
# 0 "<built-in>"
# 0 "<command-line>"
```

Figure 4. Preprocessing Command

2. Stage 2: Compilation to Assembly:

```
salman@salman-QEMU-Virtual-Machine:~/Desktop$ gcc -S program1.i -o program1.s
salman@salman-QEMU-Virtual-Machine:~/Desktop$ cat program1.s
        .arch armv8-a
        .file "program1.c"
        .text
```

Figure 5. Compilation Stage

3. Stage 3: Assembly to Object File:

```
salman@salman-QEMU-Virtual-Machine:~/Desktop$ gcc -c program1.s -o program1.o
salman@salman-QEMU-Virtual-Machine:~/Desktop$ file program1.o
program1.o: ELF 64-bit LSB relocatable, ARM aarch64, version 1 (SYSV), not strip
ped
```

Figure 6. Assembly

4) Stage 4: Linking:

```
salman@salman-QEMU-Virtual-Machine:~/Desktop$ gcc program1.o -o program1
salman@salman-QEMU-Virtual-Machine:~/Desktop$ ./program1
Hello, World!
```

Figure 7. Linking Stage

2. Program with User Input:

- The use of printf and scanf for input/output operation:

Program II

```
salman@salman-QEMU-Virtual-Machine:~/Desktop$ nano program2.c
#include <stdio.h>

int main() {
    int number;

    printf("Enter a number: ");
    scanf("%d", &number);

    printf("You entered: %d\n", number);
    return 0;
}
```

Figure 8. User input

D. COMPILATION STAGE

Program: 2

1. Stage 1: Preprocessing:

```
salman@salman-QEMU-Virtual-Machine:~/Desktop$ gcc -E program2.c -o program2.i
salman@salman-QEMU-Virtual-Machine:~/Desktop$ cat program2.i | less
salman@salman-QEMU-Virtual-Machine:~/Desktop$
```

Figure 9: Preprocessing

2. Stage 2: Compilation to Assembly:

```
salman@salman-QEMU-Virtual-Machine:~/Desktop$ gcc -S program2.i -o program2.s
salman@salman-QEMU-Virtual-Machine:~/Desktop$ cat program2.s
```

Figure 10: Compilation

3. Stage 3: Assembly to Object File:

```
salman@salman-QEMU-Virtual-Machine:~/Desktop$ gcc -c program2.s -o program.o
salman@salman-QEMU-Virtual-Machine:~/Desktop$ file program.o
program.o: ELF 64-bit LSB relocatable, ARM aarch64, version 1 (SYSV), not strip
ped
```

Figure 11: Assembly

4. Stage 4: Linking:

```
salman@salman-QEMU-Virtual-Machine:~/Desktop$ gcc program2.o -o program2
salman@salman-QEMU-Virtual-Machine:~/Desktop$ ./program2
Enter a number: 99
You entered: 99
```

Figure 12: Linking Command

3. Program with User Input:

- Find multiple format specifiers and variables:

Program III

```
salman@salman-QEMU-Virtual-Machine:~/Desktop$ nano program3.c
#include <stdio.h>

int main() {
    char name [50];
    int age;
    float height;

    printf("Enter your name: ");
    scanf("%s", name);

    printf("Enter your age: ");
    scanf("%d", &age);

    printf("Enter your height (m): ");
    scanf("%f", &height);

    printf("\n--- User Info ---\n");
    printf("Name: %s\n", name);
    printf("Age: %d years\n", age);
    printf("Height: %.2f m\n", height);

    return 0;
}
```

Figure 13: Find multiple variable

E. COMPILATION STAGE

Program: 3

1. Stage 1: Preprocessing:

```
salman@salman-QEMU-Virtual-Machine:~/Desktop$ gcc -E program3.c -o program3.i
salman@salman-QEMU-Virtual-Machine:~/Desktop$ cat program3.i | less
```

Figure 14: Preprocessing

2. Stage 2: Compilation to Assembly:

```
salman@salman-QEMU-Virtual-Machine:~/Desktop$ gcc -S program3.i -o program3.s
salman@salman-QEMU-Virtual-Machine:~/Desktop$ cat program3.s
```

Figure 15: Compilation

3. Stage 3: Assembly to Object File:

```
salman@salman-QEMU-Virtual-Machine:~/Desktop$ gcc -c program3.s -o program3.o
salman@salman-QEMU-Virtual-Machine:~/Desktop$ cat program3.o
ELF@@@
#####C#####3#####C#####
```

Figure 16: Assembly

4. Stage 4: Linking:

```
salman@salman-QEMU-Virtual-Machine:~/Desktop$ gcc program3.o -o program3
salman@salman-QEMU-Virtual-Machine:~/Desktop$ ./program3
Enter your name: Salman Ansari
Enter your age: 23
Enter your height (m): 5.6

--- User Info ---
Name: Salman Ansari
Age: 23 years
Height: 5.60 m
```

Figure 17: Linking and Run code

IV. RETURN VALUES AND EXIT STATUS

A.

i) Understanding Return Value

```
salman@salman-QEMU-Virtual-Machine:~/Desktop$ nano hello.c
salman@salman-QEMU-Virtual-Machine:~/Desktop$ gcc hello.c -o hello

#include <stdio.h>

int main() {
    printf("Program executed Successfully\n");
    return 0;
}
```

Figure 18: C-program

B. i) Checking Exit Status

```
# Run program
./hello
# Check exit status (returns 0 for success)
echo $?
# Try with return 1
# Modify program to return 1, recompile, and check:
echo $? # Should return 1 (indicating error)
```

```
salman@salman-QEMU-Virtual-Machine:~/Desktop$ ./hello
Program executed Successfully
salman@salman-QEMU-Virtual-Machine:~/Desktop$ echo $?
0
```

Figure 19: Output & Return Value

ii) Understanding Return Value

```
salman@salman-QEMU-Virtual-Machine:~/Desktop$ nano hello.c
salman@salman-QEMU-Virtual-Machine:~/Desktop$ gcc hello.c -o hello
```

```
#include <stdio.h>

int main() {
    printf("Program executed Successfully\n");
    return 1;
}
```

Figure 20: C-program

ii) Checking Exit Status

```
salman@salman-QEMU-Virtual-Machine:~/Desktop$ ./hello
Program executed Successfully
salman@salman-QEMU-Virtual-Machine:~/Desktop$ echo $?
1
```

Figure 21: Output & Return Value

V. RETURN VALUES AND EXIT STATUS

#. Understanding Concepts:

A. Question: 1

Q: What is the purpose of the `#include` directive in C programming? Provide an example of its usage.

Answer: `#include` adds a header file to the C program.

Example:

```
#include <stdio.h>
```

- It allows to use functions like `printf()` and `scanf()`.

B. Question: 2

Q: Explain the difference between the four stages of compilation. What file extension is produced at each stage?

Answer: The four compilation stages are:

- I. Preprocessing → `.i`
- II. Compilation → `.s`
- III. Assembly → `.o`
- IV. Linking → executable file

C. Question: 3

Q. What does a return value of 0 from the `main()` function indicate to the operating system? What about a non-zero return value?

Answer:

- Return 0 → Program completed successfully.
- Non-zero value → Indicates an error or different exit status.

D. Question: 4

Q: Write a C program that:

- Declares an integer variable
- Asks the user to input a value
- Displays the entered value back to the user

Answer:

```
#include <stdio.h>

int main() {
    int number;
    printf("Enter a number: ");
    scanf("%d", &number);
    printf("You entered: %d\n", number);
    return 0;
}
```

Figure 1: C-Program

E. Question: 5

Q: What GCC command would you use to:

1) Only preprocess a file without compiling?

command: `gcc -E hello.c -o hello.i`

2) Generate assembly code but not assemble it?

command: `gcc -S hello.c -o hello.s`

3) Create an object file without linking?

Command: `gcc -c hello.c -o hello.o`

F. Question: 6

Q: Explain why C is called a "compiled language" and how this differs from interpreted languages.

Answer: C is called a compiled language because its source code is converted into machine code by a compiler before execution.

An interpreted language executes code through an interpreter, usually at runtime.

CONCLUSION:

In this practical work, I learned the basic concepts of C programming and how a C program is compiled and executed. I practiced writing simple programs using `printf()` and `scanf()` and learned how to take input from the user and display the output.

I also learned about the four stages of compilation: preprocessing, compilation, assembly, and linking. By using GCC commands, I was able to see how a C source file changes into different files such as `.i`, `.s`, and `.o`, and finally into an executable program.

Another important thing I learned was the use of return values and exit status. I understood that `return 0` normally means that the program finished successfully, and `echo $?` can be used to check the exit status.

Overall, this practical helped me understand C programming and the GCC compilation process more clearly. It also gave me more practice using Linux commands and working with C programs in a Linux environment.